

ĐẶC TẢ HỆ THỐNG GIÁM SÁT XE ĐƯA ĐÓN HỌC SINH

(Viết lại từ mô tả gốc + bổ sung payload đầy đủ + sửa cấu trúc thư mục Raspberry Pi)

Version: 1.3.0 — Sửa lỗi thiết kế: RFID phụ xe **không** đặt trên Master (SPI1) như bản trước — vì học sinh và phụ xe lên xe **chung 1 cửa**, hệ thống chỉ cần **1 đầu đọc RFID duy nhất trên Slave** (đã có sẵn cho học sinh), dùng chung cho cả 2 đối tượng. Bỏ hẳn ý định thêm SPI1/module RFID thứ 2 trên Master.

1. TỔNG QUAN VAI TRÒ CÁC THÀNH PHẦN

Thành phần	Vai trò
Raspberry Pi	Gateway — không phải master, không phải nơi ra quyết định nghiệp vụ chính. Chỉ: chạy camera nhận diện khuôn mặt, chạy UI, và chuyển tiếp dữ liệu giữa STM32 Master ↔ Server.
STM32 Master	Bộ não phần cứng trên xe. Giao tiếp: UART1 (Pi), UART2 (GNSS), SPI2 (CAN qua MCP2515) với Slave, UART6 (DFPlayer). Gộp dữ liệu Slave thành 1 luồng duy nhất gửi lên Pi. Chạy FreeRTOS — xem task đề xuất ở mục 1.1.
ESP32 Slave-1 (ghế)	Đọc 16 GPIO ghế + SOS, dùng TWAI + SN65HVD230, gửi Master qua CAN.
STM32 Slave-2 (RFID+DHT11, F411CEU6)	Đặt gần cửa, đọc RFID+DHT11, gửi Master qua CAN (MCP2515 riêng).
STM32 Slave (gộp)	Thay thế cả 2 hàng trên. Đặt gần cửa lên/xuống (ưu tiên dây RFID ngắn). Gộp trong 1 node: 16 ghế qua 2× 74HC165 , SOS (ngắt ngoài), 1 đầu đọc RFID (SPI) dùng chung cho cả học sinh và phụ xe — vì cả 2 đều lên xe qua cùng 1 cửa, không cần đầu đọc riêng —, DHT11 (bit-bang) — gửi hết về Master qua 1 module MCP2515 + SN65HVD230 MCP2515 (loại module phổ biến đã tích hợp sẵn transceiver trên board, không cần thêm IC rời). Chạy FreeRTOS — xem mục 1.1.
Server (Backend + MySQL)	Lưu trữ dữ liệu chính thức, xử lý nghiệp vụ so khớp, phục vụ 3 giao diện: quản trị viên, hỗ trợ tài xế (nếu có web riêng), app phụ huynh.

Ghi chú phần cứng Slave (gộp): mỗi input của 74HC165 cần điện trở pull-up ngoài 10kΩ (IC không có pull-up nội). Board F411CEU6 giữ nguyên (không có CAN nội, vẫn cần module MCP2515 — loại module rời phổ biến ngoài thị trường đã tích hợp sẵn transceiver CAN trên cùng board, không cần mua thêm IC SN65HVD230 riêng); cân nhắc đổi F405/F407 (có bxCAN) để bỏ hẳn module MCP2515 là tối ưu thêm, không bắt buộc. Đánh đổi khi gộp: 1 lỗi phần cứng ở Slave giờ làm mất luôn cả ghế + RFID + SOS cùng lúc (trước đây 2 Slave riêng còn dự phòng lẫn nhau) — bù lại bằng 1 khung **heartbeat CAN định kỳ** (Slave gửi mỗi 1-2s), Master phát hiện mất heartbeat quá X giây thì tự báo lỗi phần cứng lên Pi/Server ngay, không chờ phát hiện gián tiếp.

1.1. Đề xuất task FreeRTOS cho từng node (mới)

Nguyên tắc chung: mỗi nguồn ngắt/I-O riêng → 1 task; ISR chỉ làm việc tối thiểu (đặt cờ/đẩy vào queue) rồi trả quyền ngay, không xử lý logic trong ISR. SOS luôn là task priority cao nhất ở cả 2 node vì liên quan an toàn tính

mạng.

STM32 Master:

Task	Priority	Việc chính	Đồng bộ
CAN_Rx	Cao nhất	Nhận khung CAN từ Slave (SOS/ghế/Rfid/DHT11/heartbeat), phân loại theo ID	ISR CAN → queue
UART_Pi	Cao	Nhận lệnh từ Pi (UART1), gửi dữ liệu cảm biến/GNSS lên Pi theo khung frame mục 3	Queue 2 chiều với các task khác
GNSS_Parse	Trung bình	Đọc UART2, parse NMEA → lat/lon/speed	Queue → UART_Pi
Rfid_Attendant	Trung bình	Đọc SPI Rfid phụ xe	Queue → UART_Pi
Audio_DFPlayer	Trung bình	Nhận lệnh phát track (map EVENT_MAIN/SUB), gửi UART6; riêng 08/001 lặp lại liên tục tới khi có lệnh hủy	Queue nhận lệnh
Heartbeat_Monitor	Thấp	Theo dõi heartbeat CAN từ Slave, báo lỗi phần cứng nếu mất quá X giây	Timer + queue

Bỏ task `Rfid_Attendant` — Master không còn đầu đọc Rfid riêng (xem lý do ở mục 1, hàng Slave gộp). Mã thẻ phụ xe giờ đến từ Slave qua CAN ID 0x200, xử lý chung trong task `CAN_Rx`.

STM32 Slave (gộp):

Task	Priority	Việc chính	Đồng bộ
SOS_Alert	Cao nhất	Gửi ngay CAN ID 0x100 khi nút SOS được nhấn, không đợi chu kỳ poll	ISR nút SOS → đánh thức task
Seat_Read	Cao	Đọc 2x74HC165 định kỳ (VD 100ms), debounce 10s, gửi CAN 0x120 khi đổi trạng thái + định kỳ	Mutex SPI/CAN dùng chung
Rfid_Read (gộp học sinh + phụ xe)	Trung bình	Đọc Rfid (SPI), gửi CAN 0x200 khi quét thẻ thành công — dùng chung 1 đầu đọc cho cả học sinh và phụ xe (cùng cửa lên xe), Server phân biệt bằng cách tra <code>rfid_code</code> thuộc bảng nào	Mutex SPI/CAN dùng chung
DHT11_Read	Thấp	Đọc nhiệt độ/độ ẩm mỗi vài giây, gửi CAN 0x210	Critical section ngắn khi bit-bang (~20-30ms) để tránh task khác chen ngang timing
Heartbeat_Send	Thấp	Gửi CAN 0x220 định kỳ 1-2s dù không có gì thay đổi	Timer

Vì `Seat_Read`, `Rfid_Read`, `DHT11_Read`, `Heartbeat_Send` đều cần gửi ra cùng 1 bộ MCP2515 (dùng chung SPI), cần 1 **mutex bảo vệ truy cập SPI/CAN controller** để tránh 2 task tranh nhau ghi cùng lúc — task nào cần gửi CAN thì `xSemaphoreTake` trước, gửi xong `xSemaphoreGive`, trừ `SOS_Alert` vẫn giữ priority cao nhất để chen ngang khi cần.

Ghi nhớ nguyên tắc xuyên suốt: **Pi không tự quyết định đúng/sai nghiệp vụ**, Pi chỉ trích xuất đặc trưng khuôn mặt tại chỗ để giảm băng thông, còn **kết quả so khớp cuối cùng là do Server trả về** (trừ trường hợp không có mạng — xem mục 7). Kiến trúc CAN đa node cũng cho phép **thêm/bớt Slave mà không ảnh hưởng phần mềm Pi/Server** — Master luôn là điểm gộp dữ liệu duy nhất phía trên.

2. LƯỒNG NGHIỆP VỤ (viết lại rõ ràng)

2.1. Đăng nhập tài xế

⚠ **Trước khi cho phép thao tác đăng nhập bất kỳ**, Pi phải đi qua bước xác nhận trạng thái phiên với Server — xem mục **2.9** (mới bổ sung). Lồng dưới đây (2.1) chỉ chạy sau khi Server xác nhận `session_state = NONE` hoặc `RESUMED` (không cần đăng nhập lại) hoặc `REQUIRE_NEW_LOGIN`.

1. Camera stream liên tục ngay khi hệ thống khởi động (màn hình chờ trước đăng nhập).
2. Tài xế nhấn nút **Login** trên UI → camera chụp ảnh → detect face → encode thành vector 128 chiều.
3. Pi đóng gói payload, gửi qua `communication` app đến Server.
4. Server so khớp vector với CSDL, trả về kết quả gồm: họ tên, số giấy phép lái xe, `result` (1 = thành công / 0 = thất bại).
5. Pi nhận phản hồi → gửi lệnh xuống STM32 Master qua UART1 → Master phát âm thanh tương ứng qua DFPlayer + hiển thị trạng thái trên UI.

2.2. Duy trì hiện diện tài xế (chống đánh tráo sau khi đăng nhập)

- Cứ mỗi **3 phút**, Pi tự động chụp lại, trích xuất đặc trưng, so với đặc trưng đã lưu lúc login (so cục bộ trên Pi, không cần hỏi Server để tiết kiệm thời gian phản hồi).
- Sai → phát cảnh báo qua loa, tăng biến đếm `mismatch_count`.
- Sai đến lần thứ **4** → gửi cảnh báo vi phạm lên Server.
- Đúng → reset `mismatch_count` về 0.

2.3. Đăng nhập / giám sát phụ xe

- Phụ xe **quẹt thẻ RFID** (đầu đọc RFID nằm ở STM32 Master, SPI dùng chung đầu đọc RFID với học sinh trên STM32 Slave — vì cả 2 lên xe qua cùng 1 cửa, không cần đầu đọc riêng) → Slave gửi CAN ID 0x200 lên Master → Master chuyển mã thẻ lên Pi qua UART1 → Pi gửi lên Server so khớp → Server tra `rfid_code` xem thuộc bảng học sinh hay phụ xe để biết đây là lượt quẹt của ai → trả kết quả → hiển thị + phát loa (Pi chỉ đóng vai trò gateway, không tự so khớp RFID).
- Giám sát hiện diện: liên tục kiểm tra **ghế số 2** (ghế số 1 = tài xế, ghế số 2 = phụ xe). Ghế trống liên tục quá **3 phút** → cảnh báo loa + tăng `absence_count`. Đủ **3 lần** → gửi vi phạm lên Server. Có người ngồi lại → reset về 0.

2.4. Học sinh lên/xuống xe

- Học sinh được xác định bằng: **mã thẻ RFID quẹt thành công + có ghế đang được ngồi tương ứng** (2 điều kiện phải khớp nhau).
- Buổi sáng (đón): học sinh cuối cùng trong tuyến đã lên xe → phát thông báo "học sinh cuối cùng đã lên xe, tài xế vui lòng quay xe về trường".
- Buổi chiều (trả): học sinh cuối cùng xuống xe (RFID học sinh cuối quẹt xuống và số học sinh trên xe = 0) → phát thông báo tương tự cho chiều về.
- Nếu số học sinh đang ngồi trên ghế > số thẻ RFID đã quẹt hợp lệ → có học sinh **quên quẹt thẻ điểm danh** → phát nhắc nhở.

2.5. Trạng thái ghế (~~ESP32 Slave~~ **STM32 Slave (gộp)** → Master → Pi)

- Slave đọc ~~16 GPIO~~ **16 ghế qua 2x74HC165 (đọc định kỳ, VD mỗi 100ms)**, gửi chuỗi dạng **`s1:0s2:1s3:0...s16:0`** về Master **ngay khi có thay đổi**, và gửi **định kỳ** để Master đối chiếu chống lỗi.
- Chống nhiễu: trạng thái ghế chỉ được xem là **thay đổi thật** nếu giữ nguyên giá trị mới liên tục **≥ 10 giây** (debounce), tránh báo sai do rung lắc/học sinh đứng dậy tạm thời.

2.6. Nút khẩn cấp (SOS)

- Nút SOS gắn ở ~~ESP32 Slave~~ **STM32 Slave (gộp)** (cùng vị trí với cảm biến ghế). Nhấn giữ lâu hoặc nhấn liên tục → Slave báo Master → Master báo Pi qua UART1 → Pi gửi khẩn cấp lên Server ngay lập tức (ưu tiên cao nhất, không qua hàng đợi). **Ở tầng firmware Slave (FreeRTOS), SOS được xử lý bằng ISR đặt cờ → đánh thức 1 task riêng có priority cao nhất trong hệ thống, gửi ngay CAN ID 0x100, không đợi tới chu kỳ đọc 74HC165 kế tiếp.**

2.7. Các tình huống nguy hiểm cần xử lý riêng (bổ sung)

a) Có học sinh trên xe nhưng KHÔNG có tài xế và KHÔNG có phụ xe Đây là tình huống nguy hiểm nhất (trẻ bị bỏ quên/tự ý lên xe không ai giám sát) — không phụ thuộc vào việc ai đã login hay chưa, chỉ cần dựa vào **cảm biến ghế thực tế**:

- Điều kiện: bất kỳ ghế học sinh nào (ghế 3–16) đang **occupied = 1**, **đồng thời** ghế 1 (tài xế) và ghế 2 (phụ xe) đều = **0**, kéo dài quá một ngưỡng ngắn (đề xuất **15–20 giây**, ngắn hơn nhiều so với debounce 10s bình thường vì đây là cảnh báo an toàn tính mạng, không phải nghiệp vụ thường).
- Xử lý: phát cảnh báo còi lớn nhất có thể (loại khác hẳn các thông báo thường), gửi cảnh báo **CRITICAL** lên Server **ngay lập tức**, không chờ, không qua hàng đợi, gửi lặp lại liên tục cho tới khi tình huống được giải quyết (có tài xế hoặc phụ xe quay lại ghế).
- Đây cũng là cảnh báo cần bật **kể cả khi xe đang tắt máy / đã kết thúc chuyến** — vì tình huống "bỏ quên trẻ trên xe" trong thực tế xảy ra chính xác lúc tài xế nghỉ chuyến đã xong và rời xe.

b) Lên/xuống xe hàng loạt tại trường (không quét thẻ RFID) Thực tế đúng như anh mô tả: học sinh chỉ cần quét thẻ ở **điểm đón tại nhà** (buổi sáng) và **điểm trả tại nhà** (buổi chiều) — còn lúc đến/rời trường là **xuống/lên hàng loạt cùng lúc**, không quét thẻ từng em. Cần định nghĩa rõ 4 giai đoạn trong ngày (**trip_phase**) để hệ thống không hiểu nhầm "hàng loạt không quét thẻ" là lỗi:

Giai đoạn	Mô tả	Có cần quét thẻ từng học sinh?
PICKUP	Xe đi đón, dừng từng nhà	✅ Có — quét thẻ khi lên xe
ARRIVE_SCHOOL	Xe đến trường, tất cả xuống cùng lúc	❌ Không — xử lý theo lô (bulk)
DEPART_SCHOOL	Cuối buổi học, tất cả lên xe cùng lúc để về	❌ Không — xử lý theo lô (bulk)
DROPOFF	Xe đi trả, dừng từng nhà	✅ Có — quét thẻ khi xuống xe

- Nhận diện chuyển giai đoạn **ARRIVE_SCHOOL / DEPART_SCHOOL**: dựa vào (1) vị trí GNSS nằm trong bán kính trường (geofence toạ độ trường, lấy từ CSDL) **và** (2) nhiều ghế thay đổi trạng thái gần như đồng thời (VD: >70% ghế đang có học sinh đổi trạng thái trong cùng khung ~10–15 giây).
- Khi phát hiện bulk alight/board: **đối chiếu số lượng**, không cần RFID — so **số ghế vừa trống** (hoặc vừa có người) với **số học sinh đang được ghi nhận là đã lên xe từ đầu tuyến PICKUP** (danh sách đã quét thẻ buổi sáng). Nếu khớp → ghi nhận sự kiện bulk bình thường, gửi lên Server để log, không cảnh báo. Nếu **không khớp** (VD: 14 học sinh đã lên nhưng chỉ 13 ghế trống khi tới trường) → đây chính là dấu hiệu **có khả năng bỏ quên học sinh trên xe** → escalate như tình huống (a) ở trên ngay lập tức, không chờ.

- Buổi chiều, lúc **DROPOFF**: hệ thống quay lại yêu cầu quét thẻ **từng em** khi xuống — vì lúc này mỗi điểm dừng là một nhà khác nhau, cần xác nhận đúng em xuống đúng nhà (phục vụ luôn thông báo cho app phụ huynh).
- **Âm thanh nhắc kiểm tra xe**: đúng lúc chuyển giai đoạn **ARRIVE_SCHOOL** (bulk alight khớp số) và lúc học sinh cuối cùng xuống xe ở **DROPOFF** (kết thúc chuyến chiều), Master phát lời nhắc **"kiểm tra kỹ còn học sinh nào trên xe không"** trước khi tài xế/phụ xe rời xe — xem mã 05/006 và 05/004 đã cập nhật ở mục 3.2. Đây chính là 2 thời điểm cuối cùng còn kịp phát hiện trước khi mọi người rời xe, nên đặt lời nhắc ở đây là hợp lý nhất.

2.8. Ba giao diện của hệ thống

1. **UI trên Pi (tài xế)**: trước đăng nhập — stream camera, nút login, nút SOS, trạng thái wifi. **Ngay lúc khởi động, có thể hiện trạng thái "Đang chờ xác nhận từ trung tâm" nếu Server phát hiện phiên cũ chưa đóng (xem mục 2.9)** — nút login tạm khoá cho tới khi Server trả lời rõ ràng. Sau đăng nhập — sơ đồ ghế trống/đầy, trạng thái phiên làm việc, trạng thái tài xế/phụ xe có mặt trên xe, số học sinh hiện tại, bản đồ, danh sách học sinh cần đón. Nút **Logout chỉ bật được khi**: không còn học sinh trên xe **và** phụ xe đã đăng xuất — nếu cố logout mà chưa đủ điều kiện, hệ thống báo rõ lý do (còn học sinh / còn phụ xe) và **logout thất bại**. Logout cũng phải trích xuất khuôn mặt để xác nhận đúng là tài xế.
2. **Giao diện quản trị (Server, MySQL)**: quản lý Xe / Tài xế / Phụ xe / Học sinh / Enroll thiết bị mới, dashboard bản đồ GNSS (OpenStreetMap), trạng thái tài xế gồm **active** (hoạt động bình thường) / **pending** (chưa xác thực khuôn mặt) / **suspended** (bị tạm hoãn do vi phạm nhiều lần — chỉ **active** mới login được, các trạng thái khác **đều từ chối đăng nhập**). Luồng enroll: nhập thông tin → lưu MySQL (status = pending) → xác thực khuôn mặt → chuyển **active** → cập nhật vector đặc trưng vào MySQL. **Thêm: khi phát hiện phiên cũ tồn đọng lúc xe khởi động lại (mục 2.9), Dashboard hiện popup/thông báo riêng với 2 nút Agree / Disagree cho quản trị viên xử lý — không tự động quyết định.**
3. **App phụ huynh**: đăng ký cho học sinh nghỉ học (xe sẽ bỏ qua điểm dừng đó), nhận thông báo "xe đang đến" theo thứ tự dừng thực tế của tuyến (nếu bạn A đã lên, bạn B là điểm dừng kế tiếp sẽ được báo).

2.9. Xác nhận trạng thái phiên khi khởi động lại (mới bổ sung)

Vấn đề: nếu Pi mất nguồn giữa chừng (VD: đang trong phiên làm việc, xe tắt máy đột ngột hoặc lỗi nguồn) rồi khởi động lại, Pi **không thể tự biết chắc** phiên trước đó đã kết thúc đúng cách hay chưa — bộ nhớ RAM mất, trạng thái cục bộ không đáng tin. Nguyên tắc: **Pi tuyệt đối không tự phán đoán, luôn hỏi Server và tin tưởng hoàn toàn kết quả trả về.**

Luồng xử lý:

1. Ngay khi Pi khởi động (hoặc kết nối lại mạng sau một khoảng ngắt quãng), **trước khi cho phép bất kỳ thao tác đăng nhập nào**, Pi gửi **vehicle_status_request** (type=21) lên Server, kèm biển số xe (**vid**).
2. Server tra trong CSDL: xe này (theo biển số) có đang tồn tại một **phiên làm việc chưa được đóng đúng cách** hay không (VD: có driver_id đã login mà chưa có logout tương ứng).
 - **Không có phiên nào tồn đọng** → Server trả **session_state = 0 (NONE)** → Pi cho vào màn hình đăng nhập bình thường, không cần chờ gì thêm (trường hợp phổ biến nhất — khởi động buổi sáng bình thường).
 - **Có phiên tồn đọng** → Server trả **session_state = 1 (PENDING_CONFIRM)** kèm thông tin tài xế/phụ xe của phiên cũ để hiển thị cho quản trị viên tham khảo. Đồng thời, Server **hiện lên Dashboard 1 yêu cầu xác nhận** với 2 lựa chọn **Agree / Disagree** cho quản trị viên xử lý. Lúc này UI trên Pi hiển thị trạng thái **"Đang chờ xác nhận từ trung tâm"**, nút đăng nhập tạm khoá (không phải lỗi, chỉ là đang chờ).
3. Quản trị viên xem xét (VD: gọi điện xác minh với tài xế, hoặc biết chắc xe chỉ bị mất điện thoáng qua) rồi chọn trên Dashboard:

- **Agree** → Server gửi tiếp `vehicle_status_response` (type=22) với `session_state = 2 (RESUMED)`, kèm đầy đủ thông tin tài xế/phụ xe của phiên cũ → Pi **khôi phục lại trạng thái đã đăng nhập ngay**, dùng đúng thông tin Server gửi, **không yêu cầu chụp lại khuôn mặt**.
 - **Disagree** → Server gửi `session_state = 3 (REQUIRE_NEW_LOGIN)` → Pi mở khoá màn hình đăng nhập, tài xế phải đăng nhập lại từ đầu như luồng 2.1 bình thường (phiên cũ coi như bị đóng, Server tự đánh dấu logout ép buộc phiên cũ trong CSDL).
4. Trong lúc chờ (bước 2 → 3), nếu Pi mất kết nối lại thì gửi lại `vehicle_status_request` mỗi khi có mạng, không tự ý coi như "đã được Agree" nếu chưa nhận được `session_state = 2` rõ ràng từ Server.

Payload:

```
// Pi → Server, ngay lúc khởi động
{ "type": 21, "ts": 1752720000, "vid": "29B12345", "data": {}, "sig": "..."} }
```

```
// Server → Pi, phản hồi ngay lập tức (có phiên tồn đọng)
{
  "type": 22, "ts": 1752720001, "vid": "29B12345",
  "data": {
    "session_state": 1,
    "existing_driver_id": "GV0012", "existing_driver_name": "Nguyễn Văn A",
    "existing_attendant_id": "PX0003"
  },
  "sig": "..."} }
```

```
// Server → Pi, sau khi admin bấm Agree trên Dashboard
{
  "type": 22, "ts": 1752720340, "vid": "29B12345",
  "data": {
    "session_state": 2,
    "driver_id": "GV0012", "driver_full_name": "Nguyễn Văn A", "license_number":
    "123456789",
    "attendant_id": "PX0003", "attendant_full_name": "Trần Thị B"
  },
  "sig": "..."} }
```

```
// Server → Pi, sau khi admin bấm Disagree
{ "type": 22, "ts": 1752720340, "vid": "29B12345", "data": { "session_state": 3 },
  "sig": "..."} }
```

`session_state`: 0=NONE, 1=PENDING_CONFIRM, 2=RESUMED, 3=REQUIRE_NEW_LOGIN. Topic dùng: `.../vehicle/status/request` và `.../vehicle/status/response` (bổ sung vào bảng mục 4.2). Type 21 thuộc nhóm **KHÔNG cache** (mục 7.1) — cần phản hồi thật để Pi biết đường xử lý màn hình chờ, không được để trong hàng đợi ngoại tuyến.

⚠ **Quan trọng:** `session_state` chỉ nói về **danh tính tài xế/phụ xe**, hoàn toàn tách biệt với **trạng thái chuyển đi** (đang ở giai đoạn nào, đã bao nhiêu học sinh lên xe) — trạng thái chuyển đi được lưu riêng ở bảng `TripSession` (mục 8) và luôn được gửi kèm trong cùng payload `type=22`, bất kể kết quả là Agree hay Disagree, để tránh trường hợp tài xế đăng nhập lại làm hệ thống tưởng nhầm là "chuyển đi mới" rồi báo động giả bỏ quên trẻ.

3. GIAO THỨC UART GIỮA RASPBERRY PI ↔ STM32 MASTER

Áp dụng đúng phong cách khung dữ liệu anh đã dùng khi port driver DFPlayer (có checksum, giữ định dạng frame cố định) — dùng lại cho toàn bộ giao tiếp Pi ↔ Master để đồng bộ 1 chuẩn duy nhất, tránh mỗi loại lệnh một kiểu.

Lưu ý: dù phía dưới Master có ~~2 Slave (ghế + RFID/DHT11)~~ **1 Slave (đã gộp ghế + RFID/DHT11 + SOS)** giao tiếp qua CAN, Master vẫn là điểm gộp duy nhất — khung UART gửi lên Pi không đổi, Pi không cần biết có bao nhiêu STM32 phía sau.

3.0. Bảng CAN ID giữa Master ↔ Slave (gộp) (nội bộ, không liên quan Pi/Server)

Số ID càng nhỏ càng được ưu tiên khi tranh chấp bus. **Bảng ID giữ nguyên dù đã gộp về 1 node vật lý — mỗi ID vẫn là 1 loại message riêng, Master xử lý y như trước.**

Ưu tiên	Nội dung	CAN ID
Cao nhất	Nút SOS khẩn cấp	0x100
Cao	Trạng thái 16 ghế	0x120
Trung bình	Mã RFID vừa quét (dùng chung học sinh + phụ xe)	0x200
Thấp	Dữ liệu DHT11 (nhiệt độ/độ ẩm)	0x210
Thấp nhất	Heartbeat định kỳ (Master phát hiện Slave mất kết nối)	0x220

3.1. Cấu trúc khung (frame)

STX	EVENT_MAIN	EVENT_SUB	LEN	DATA[LEN]	CHECKSUM	ETX
0xAA	1 byte	1 byte	1B	N bytes	1B	0x55

- **STX** = 0xAA, **ETX** = 0x55 (cố định, đánh dấu đầu/cuối khung).
- **EVENT_MAIN**, **EVENT_SUB**: **dùng lại chính bảng mã âm thanh anh đã định nghĩa** (01/001, 05/003...) → Master chỉ cần map thẳng sang thư mục/track DFPlayer, không cần Pi gửi thêm lệnh phát nhạc riêng.
- **DATA**: dữ liệu phụ (VD: mã tài xế, mã RFID) — có thể rỗng (**LEN** = 0).
- **CHECKSUM**: XOR toàn bộ byte từ **EVENT_MAIN** đến hết **DATA** (giữ nguyên logic checksum anh đã dùng ở driver DFPlayer STM32→ESP32).

3.2. Bảng sự kiện (EVENT_MAIN / EVENT_SUB) — giữ nguyên bảng mã anh đã thiết kế

Mã	Sự kiện	Track loa
01/001	Tài xế login thành công	01/001
01/002	Tài xế login thất bại	01/002

Mã	Sự kiện	Track loa
02/001	Tài xế logout thành công	02/001
02/002	Tài xế logout thất bại	02/002
03/001	Phụ xe login thành công	03/001
03/002	Phụ xe login thất bại	03/002
04/001	Phụ xe logout thành công	04/001
04/002	Phụ xe logout thất bại	04/002
05/001	Học sinh quẹt thẻ thành công	05/001
05/002	Mã RFID không hợp lệ / lỗi quẹt thẻ	05/002
05/003	Học sinh cuối cùng đã lên xe (chiều đón)	05/003
05/004	Học sinh cuối cùng đã xuống xe tại nhà (chiều trả) — nội dung: " <i>Học sinh cuối cùng đã xuống xe, bác tài và phụ xe vui lòng kiểm tra lại xe trước khi kết thúc chuyến</i> "	05/004
05/005	Có học sinh trên ghế chưa quẹt thẻ	05/005
06/001	Khuôn mặt tài xế không khớp lúc kiểm tra định kỳ	06/001
06/002	Phụ xe check-in sau khi đã có học sinh trên xe	06/002
06/003	Phụ xe không có mặt tại ghế	06/003
07/001 (mới)	Nhấn nút SOS khẩn cấp	07/001
05/006 (mới)	Xuống xe hàng loạt tại trường, đã khớp số lượng (buổi sáng) — nội dung: " <i>Tất cả học sinh đã xuống xe, bác tài và phụ xe hãy kiểm tra thật kỹ xem còn học sinh nào trên xe không trước khi xuống xe nhé</i> "	05/006
05/007 (mới)	Lên xe hàng loạt tại trường (bulk, khớp số lượng) — chiều về, không cần thoại nhắc riêng, chỉ log	05/007
08/001 (mới — ưu tiên cao nhất)	CẢNH BÁO: có học sinh trên xe nhưng không có tài xế/phụ xe (ngghi bỏ quên trẻ)	08/001

Đề xuất thêm 07/001, 05/006, 05/007, 08/001. Mã **08/001** nên dùng file âm thanh khác hẳn (còi lớn, liên tục) so với các mã còn lại, vì đây là cảnh báo an toàn tính mạng, không phải thông báo nghiệp vụ thông thường — Master nên **lặp lại phát cảnh báo này liên tục** cho tới khi Pi gửi lệnh huỷ (khi có người quay lại ghế tài xế/phụ xe), chứ không phát 1 lần như các mã khác.

Hai mốc phát âm thanh nhắc kiểm tra xe (theo yêu cầu bổ sung): (1) lúc xuống hàng loạt tại trường buổi sáng → phát **05/006**; (2) lúc học sinh cuối cùng xuống xe tại nhà buổi chiều (kết thúc tuyến trả) → phát **05/004** với nội dung đã cập nhật ở trên. Cả 2 đều là lời nhắc **kiểm tra xe trước khi tài xế/phụ xe rời xe** — đúng lúc và đúng nội dung nhất để phòng bỏ quên trẻ, vì đây là 2 thời điểm cuối cùng còn có thể phát hiện kịp trước khi mọi người rời xe.

3.3. Ví dụ khung thực tế

Login tài xế thành công, mã tài xế **GV0012** (6 ký tự ASCII):

```
AA 01 01 06 47 56 30 30 31 32 <CHECKSUM> 55
  L01 L01 Llen=6 L—"GV0012"—
```

3.4. Chiều ngược lại (Master → Pi)

Cùng 1 khung frame, dùng cho: dữ liệu GNSS, tốc độ, chuỗi trạng thái ghế (**s1:s2:1...s16:0**), mã RFID vừa quét, trạng thái nhấn SOS. Ví dụ **EVENT_MAIN = 0xF0** (dải riêng dùng cho dữ liệu cảm biến, không trùng dải sự kiện âm thanh 01–07):

EVENT_MAIN	Ý nghĩa	DATA
0xF0	Cập nhật GNSS + tốc độ	lat, lon, speed (chuỗi ASCII)
0xF1	Cập nhật trạng thái ghế	s1:s2:1...s16:0
0xF2	Mã RFID vừa quét (học sinh/phụ xe)	mã thẻ ASCII
0xF3	Nút SOS được nhấn	0x01
0xF4 (mới)	Hủy cảnh báo 08/001 (Pi gửi xuống Master khi tài xế/phụ xe đã quay lại ghế)	rỗng

4. ĐẶC TẢ MQTT — RASPBERRY PI ↔ SERVER

4.0. Chuẩn hoá **type** bằng số nguyên (thay vì chuỗi dài)

Đúng như anh nói — để chuỗi dài như "**driver_login_request**" trong payload vừa tốn băng thông vừa không cần thiết, vì thiết bị và Server **đã biết trước với nhau ý nghĩa từng số** thông qua bảng dưới đây (giống hệt cách anh đã làm với mã âm thanh 01/002...). Bảng này là **nguồn duy nhất** (single source of truth) — cả code Server và code Pi đều import chung 1 file enum, không ai được tự định nghĩa số riêng.

type	Ý nghĩa
1	driver_login_request
2	driver_login_response
3	driver_logout_request
4	driver_logout_response
5	driver_presence_violation
6	attendant_login_request
7	attendant_login_response
8	attendant_logout_request
9	attendant_logout_response
10	attendant_presence_violation

type	Ý nghĩa
11	student_scan
12	student_event (dùng kèm event_code số: 503, 504, 505, 506, 507 — lấy từ mã UART bỏ dấu /)
13	seat_status
14	telemetry
15	emergency_sos
16	child_alone_alert — CRITICAL, mục 2.7(a)
17	driver_enroll
18	driver_enroll_ack
19	heartbeat
20	server_command (Server → Pi) — envelope lệnh chung , luôn có trường action để phân biệt loại lệnh cụ thể bên trong (VD: action: "start_stream" , action: "stop_stream" , action: "force_logout" , action: "update_roster" ...). Không tạo type riêng cho từng loại lệnh — tất cả lệnh điều khiển từ Server đều đi qua type=20, phân biệt nhau bằng action .
21 (mới)	vehicle_status_request (Pi → Server) — hỏi trạng thái phiên hiện tại của xe khi Pi khởi động/kết nối lại
22 (mới)	vehicle_status_response (Server → Pi) — Pi tin tưởng hoàn toàn kết quả này, không tự phán đoán
23 (mới)	student_scan_ack (Server → Pi) — phản hồi ngay sau mỗi lần quét thẻ, kèm điểm đến/học sinh tiếp theo để hiển thị lên UI (card "ĐIỂM ĐẾN")

Trường **message** (chuỗi tiếng Việt hiển thị) **bỏ khỏi payload thiết bị gửi lên** — Server tự sinh nội dung hiển thị dựa theo **type/event_code**, thiết bị không cần gửi text.

4.0b. Bảo toàn gói tin bằng PSK (chữ ký HMAC)

Thêm PSK là hướng ổn — với thiết bị nhúng, dùng **HMAC-SHA256 với khoá PSK** là đủ nhẹ mà vẫn chống được giả mạo/sửa gói tin, không cần đến chứng chỉ bất đối xứng nặng. Thiết kế:

- Mỗi xe có 1 **DEVICE_PSK** riêng (không dùng chung 1 key cho tất cả xe — nếu 1 xe bị lộ key, xe khác không ảnh hưởng), lưu trong **.env** của Pi và lưu tương ứng phía Server (key vault/DB, không lưu dạng plaintext).
- Mọi payload đều được bọc trong "envelope" chung:

```
{
  "type": 2,
  "ts": 1752732724,
  "vid": "29B12345",
  "data": { "...nội dung riêng theo type...": "..."},
  "sig": "8f1a3c...64 ký tự hex"
}
```

- sig** = HMAC_SHA256(PSK, "{type}|{ts}|{vid}|{json(data)}")

- Server tính lại HMAC bằng PSK của đúng xe đó (**vid**) rồi so sánh — sai chữ ký thì **loại bỏ gói tin**, không xử lý.
- **ts** (unix timestamp) dùng thêm để **chống replay**: Server từ chối gói tin nếu $|server_time - ts| > 30s$ (trừ gói tin đang được đẩy bù từ hàng đợi ngoại tuyến — xem mục 7, các gói đó có cờ riêng nên không bị chặn bởi luật 30s).
- Nên có kế hoạch xoay vòng PSK định kỳ (VD: đổi mỗi 3–6 tháng) qua lệnh **server_command**, không bắt buộc ngay trong đồ án nhưng nên ghi vào phần "hướng phát triển".

4.1. Quy ước topic

```
schoolbus/{vehicle_id}/<nhóm>/<sự_kiện>
```

{vehicle_id} = biển số xe, lấy từ file **.env** (mục 5).

4.2. Bảng topic đầy đủ

Topic	Chiều	Mục đích
.../driver/login	Pi → Server	Gửi vector khuôn mặt xin đăng nhập
.../driver/login/ack	Server → Pi	Trả kết quả login
.../driver/logout	Pi → Server	Gửi vector khuôn mặt xin đăng xuất
.../driver/logout/ack	Server → Pi	Trả kết quả logout (có thể result:0 kèm lý do)
.../driver/presence_violation	Pi → Server	Báo sai khuôn mặt 4 lần liên tiếp
.../attendant/login	Pi → Server	Gửi mã RFID phụ xe
.../attendant/login/ack	Server → Pi	Trả kết quả
.../attendant/logout / .../attendant/logout/ack	2 chiều	Tương tự
.../attendant/presence_violation	Pi → Server	Ghế phụ xe trống 3 lần
.../student/scan	Pi → Server	Mỗi lần học sinh quẹt thẻ
.../student/scan/ack (mới, <i>type=23</i>)	Server → Pi	Trả điểm đến tiếp theo (đã loại học sinh nghỉ) để hiển thị UI
.../student/event	Pi → Server	Sự kiện tổng hợp: học sinh cuối lên/xuống, cảnh báo thiếu quẹt thẻ

Topic	Chiều	Mục đích
.../seat/status	Pi → Server	Trạng thái 16 ghế (đã debounce)
.../telemetry	Pi → Server	GNSS, tốc độ, số học sinh hiện tại — gửi định kỳ (đề xuất 5–10s/lần)
.../emergency/sos	Pi → Server	Khẩn cấp — QoS 2, gửi ngay
.../emergency/child_alone (mới, type=16)	Pi → Server	Ưu tiên cao nhất — nghi bỏ quên học sinh trên xe, xem 2.7(a)
.../driver/enroll	Pi → Server	Kết quả enroll khuôn mặt tài xế mới tại xe
.../driver/enroll/ack	Server → Pi	Xác nhận đã lưu, chuyển status → active
.../system/heartbeat (đề xuất thêm)	Pi → Server	Pi còn sống, phát hiện mất kết nối xe
.../vehicle/status/request (mới, type=21)	Pi → Server	Hỏi trạng thái phiên khi khởi động — xem mục 2.9
.../vehicle/status/response (mới, type=22)	Server → Pi	Trả session_state (NONE/PENDING_CONFIRM/RESUMED/REQUIRE_NEW_LOGIN) — Pi tin tưởng tuyệt đối
.../command (đề xuất thêm)	Server → Pi	Server chủ động ra lệnh (type=20, phân biệt bằng action: VD force_logout, update_roster, start_stream, stop_stream)

4.3. Payload chi tiết (đã bọc envelope PSK + type số — mục data là phần riêng theo từng loại)

a) Đăng nhập tài xế — request (type=1)

```
{
  "type": 1, "ts": 1752732723, "vid": "29B12345",
  "data": { "face_vector": [0.0123, -0.0456, "...128 phần tử..."] },
  "sig": "..."
```

b) Đăng nhập tài xế — response (type=2, Server → Pi)

```
{
  "type": 2, "ts": 1752732724, "vid": "29B12345",
  "data": {
    "driver_id": "GV0012", "result": 1,
    "full_name": "Nguyễn Văn A", "license_number": "123456789", "license_class": "B2"
  },
```

```
"sig": "..."  
}
```

c) Cảnh báo vi phạm hiện diện (type=5)

```
{  
  "type": 5, "ts": 1752733200, "vid": "29B12345",  
  "data": { "driver_id": "GV0012", "mismatch_count": 4 },  
  "sig": "..."  
}
```

d) Đăng nhập phụ xe (type=6 request / type=7 response)

```
{ "type": 6, "ts": 1752732840, "vid": "29B12345",  
  "data": { "rfid_code": "04A3F1B2" }, "sig": "..."} }
```

```
{ "type": 7, "ts": 1752732841, "vid": "29B12345",  
  "data": { "attendant_id": "PX0003", "result": 1, "full_name": "Trần Thị B" }, "sig":  
  "..."} }
```

e) Học sinh quét thẻ (type=11) — thêm trường `trip_phase` (1=PICKUP, 4=DROPOFF; xem mục 2.7b) vì chỉ 2 giai đoạn này mới có quét thẻ cá nhân

```
{  
  "type": 11, "ts": 1752733102, "vid": "29B12345",  
  "data": { "rfid_code": "1A2B3C4D", "seat_number": 5, "result": 1, "trip_phase": 1 },  
  "sig": "..."  
}
```

e2) Phản hồi điểm đến tiếp theo (type=23, Server → Pi) — đúng như anh phát hiện qua giao diện: sau **mỗi lần quét thẻ thành công**, Server tính lại thứ tự điểm dừng còn lại (đã loại bỏ học sinh đăng ký nghỉ qua app phụ huynh — mục 2.8, ý 3) và trả về điểm đến kế tiếp để đổ vào card "ĐIỂM ĐẾN" trên UI:

```
{  
  "type": 23, "ts": 1752733103, "vid": "29B12345",  
  "data": {  
    "next_student_id": "HS0042",  
    "next_student_name": "Nguyễn Văn Lộc",  
    "next_address": "LK6D - Nguyễn Văn Lộc",  
    "next_lat": 21.0105,  
    "next_lon": 105.8501,  
    "next_route_geometry": "encoded_polyline_hoac_mang_toa_do...",  
    "is_last_student_picked": false,  
    "students_onboard": 9,  
    "students_remaining": 5  
  }  
}
```

```

},
"sig": "...
}

```

Chỉ áp dụng cho giai đoạn **PICKUP/DROPOFF** (có điểm dừng riêng từng nhà). Ở

ARRIVE_SCHOOL/DEPART_SCHOOL (bulk, mục 2.7b) không có "điểm đến kế tiếp" theo từng em, Server có thể bỏ trống hoặc trả **next_address**: "Trường học" cố định. Nếu học sinh kế tiếp trong danh sách đã đăng ký nghỉ hôm đó, Server **tự động bỏ qua** trong lúc tính **next_student_id**, Pi không cần biết ai đã nghỉ, chỉ hiển thị đúng người kế tiếp thật sự cần đón/trả.

Bổ sung — routing động + học sinh cuối cùng (mới):

- next_lat/next_lon**: toạ độ điểm đến kế tiếp — do quản trị viên **chọn bằng pin trên bản đồ** lúc đăng ký học sinh (mục 8.4b), không chỉ dựa vào geocode tự động từ địa chỉ nhập tay.
- next_route_geometry**: tuyến đường Server **tính động ngay tại thời điểm quét thẻ** (gọi routing engine — OSRM/ORS — từ vị trí GPS hiện tại của xe tới **next_lat/next_lon**), KHÔNG dùng lại tuyến tính đã nạp sẵn từ đầu chuyến (mục 8.5 cũ). Pi chỉ vẽ lại đường này lên bản đồ, không tự tính toán gì — giữ đúng nguyên tắc Pi là gateway hiển thị.
- is_last_student_picked**: chỉ có ý nghĩa ở **PICKUP**. **true** khi học sinh vừa quét là **học sinh cuối cùng cần đón** trong tuyến (học sinh cuối trong CSDL, hoặc học sinh trước đó nếu học sinh cuối đã đăng ký nghỉ). Khi **true**, **next_lat/next_lon/next_address** là toạ độ **trường** (không phải nhà học sinh nào nữa) — Pi hiển thị "đang về trường" thay vì "điểm đón tiếp theo", và **không** coi việc học sinh rời ghế lúc này là bất thường (vẫn đang trên đường về, ghế sẽ trống dần khi tới **ARRIVE_SCHOOL**).
- Dự phòng khi mất mạng/routing API lỗi lúc quét thẻ**: Pi giữ nguyên polyline đang hiển thị (không xóa trắng bản đồ), chỉ cập nhật icon xe theo GPS như bình thường cho tới khi nhận được **next_route_geometry** mới.

f) Sự kiện học sinh tổng hợp (type=12) — **event_code** số nguyên lấy từ mã UART bỏ dấu /: 503=hs cuối lên, 504=hs cuối xuống, 505=hs ngồi mà chưa quét, 506=bulk xuống ở trường (khớp số), 507=bulk lên ở trường (khớp số)

```

{
  "type": 12, "event_code": 503, "ts": 1752733810, "vid": "29B12345",
  "data": { "students_onboard": 14 },
  "sig": "...
}

```

```

{
  "type": 12, "event_code": 506, "ts": 1752734500, "vid": "29B12345",
  "data": { "seats_emptied": 14, "expected_boarded": 14, "match": true },
  "sig": "...
}

```

g) Trạng thái ghế (type=13)

```

{
  "type": 13, "ts": 1752733140, "vid": "29B12345",

```

```

    "data": { "seats":
    { "1":1, "2":1, "3":1, "4":0, "5":1, "6":0, "7":0, "8":0, "9":0, "10":0, "11":0, "12":0, "13":0, "14":0, "15":0, "16":0 } },
    "sig": "...
  }

```

h) Telemetry định kỳ (type=14)

```

{
  "type": 14, "ts": 1752733145, "vid": "29B12345",
  "data": { "lat": 21.0021, "lon": 105.8462, "speed_kmh": 32.5, "students_onboard": 14 },
  "sig": "...
}

```

i) Khẩn cấp SOS (type=15)

```

{
  "type": 15, "ts": 1752733540, "vid": "29B12345",
  "data": { "lat": 21.0021, "lon": 105.8462, "triggered_by": 1, "seat_number": 7 },
  "sig": "...
}

```

i2) CẢNH BÁO bỏ quên học sinh — không tài xế/phụ xe (type=16, ưu tiên tuyệt đối, xem 2.7a)

```

{
  "type": 16, "ts": 1752733600, "vid": "29B12345",
  "data": { "occupied_student_seats": [5, 9], "driver_seat": 0, "attendant_seat": 0,
  "duration_sec": 18 },
  "sig": "...
}

```

j) Enroll tài xế mới (type=17)

```

{
  "type": 17, "ts": 1752730200, "vid": "29B12345",
  "data": { "driver_id": "GV0015", "face_vector": [0.021, -0.033, "...128 phần tử..."] },
  "sig": "...
}

```

k) Yêu cầu điều khiển Stream Video từ Server (type=20, Server → Pi)

Điều kiện tiên quyết — chỉ hoạt động khi đã đăng nhập: trước khi login, camera đã stream **nội bộ** (chỉ hiển thị tại chỗ trên UI Pi để tài xế thấy mặt mình lúc đăng nhập, **không đẩy ra ngoài mạng**). Server **chỉ được phép gửi start_stream** khi Pi đang ở trạng thái đã login (**driver_id** khác null trong **TripSession/session** hiện tại) — nếu

Server gửi `start_stream` lúc chưa có ai login, Pi phải từ chối, trả `stream_status: "error"` kèm lý do `"no_active_driver_session"`.

Kiến trúc camera bắt buộc — 1 luồng chụp dùng chung: Raspberry Pi CSI camera chỉ cho 1 tiến trình giữ quyền độc quyền cùng lúc, nên **không được để mỗi module tự mở camera riêng** (login/logout, `presence_monitor` check định kỳ mục 2.2, và stream ra Server đều cần đọc camera). Thiết kế đúng: 1 **capture thread duy nhất** trong `apps/camera/`, mở camera 1 lần lúc khởi động, đẩy frame liên tục vào 1 buffer/queue dùng chung trong bộ nhớ — mọi module khác chỉ **đọc** từ buffer này (không mở lại thiết bị camera), tránh xung đột "device busy". Nhờ vậy, `presence_monitor` (mục 2.2) chạy hoàn toàn độc lập, không bị ảnh hưởng bởi việc đang stream hay không, và ngược lại.

Khi quản trị viên tại Backend Dashboard muốn xem camera trực tuyến từ xe, Server sẽ gửi lệnh này xuống để kích hoạt Pi đẩy luồng video (thông qua RTP/RTSP) — Pi lấy frame trực tiếp từ buffer chung, không mở thêm kết nối camera mới:

- **Bật stream video (Start Stream):**

```
{
  "type": 20, "ts": 1752734000, "vid": "29B12345",
  "data": {
    "action": "start_stream",
    "protocol": "rtp",                // "rtp" hoặc "rtsp"
    "stream_host": "192.168.0.107",  // IP Server Backend nhận luồng
    "stream_port": 5000,             // Port UDP nhận luồng RTP
    "resolution": "640x480",         // Độ phân giải stream
    "fps": 15,                       // Số khung hình trên giây
    "bitrate": "800k"                // Băng thông tối đa (tối ưu hóa mạng di động)
  },
  "sig": "...",
}
```

- **Tắt stream video (Stop Stream — do Server chủ động yêu cầu dừng):**

```
{
  "type": 20, "ts": 1752734500, "vid": "29B12345",
  "data": {
    "action": "stop_stream"
  },
  "sig": "...",
}
```

- **Phản hồi từ Pi báo trạng thái Stream (type=20 ACK, Pi → Server):**

```
{
  "type": 20, "ts": 1752734002, "vid": "29B12345",
  "data": {
    "action": "stream_status",
    "status": "active",              // "active" (đang stream), "inactive" (đã tắt),
    "error": "lỗi cấu hình/phần cứng/no_active_driver_session"
  }
}
```

```

    "message": "Streaming video successfully to 192.168.0.107:5000",
    "stream_url": "rtp://192.168.0.107:5000"
  },
  "sig": "...
}

```

Trường hợp đặc biệt — tài xế logout trong lúc đang stream: Pi **không chờ Server ra lệnh `stop_stream`**, mà **tự chủ động dừng** ngay khi xác thực logout hợp lệ, theo đúng thứ tự sau (tránh việc Server phải tự đoán qua im lặng/timeout — dễ hiểu nhầm nếu chỉ mất mạng thoáng qua):

1. Tài xế nhấn Logout → trích xuất khuôn mặt xác nhận đúng người (mục 2.8, mục 1).
2. Nếu hợp lệ **và** đang có stream hoạt động → Pi tự dừng đẩy luồng RTP/RTSP ra ngoài, đồng thời chủ động gửi báo nhận rõ ràng lên Server (không im lặng chờ Server tự suy luận):

```

{
  "type": 20, "ts": 1752736000, "vid": "29B12345",
  "data": { "action": "stream_stopped", "reason": "driver_logout" },
  "sig": "...
}

```

3. Server nhận `stream_stopped` → cập nhật lại giao diện Dashboard về trạng thái "không có stream" ngay, không cần chờ timeout.
4. Pi hoàn tất logout, camera **quay lại chế độ stream nội bộ** (chỉ hiển thị tại chỗ cho màn hình chờ) — vì vẫn dùng chung 1 buffer capture thread như trên, không cần tắt/mở lại phần cứng camera, chỉ đơn giản là ngừng đẩy frame ra mạng.

5. FILE `.env` ĐỀ XUẤT

```

VEHICLE_ID=29B-123.45
CHASSIS_NUMBER=RE123456789
MQTT_BROKER_HOST=broker.example.com
MQTT_BROKER_PORT=8883
MQTT_USERNAME=pi_29B12345
MQTT_PASSWORD=changeme
SERVER_API_BASE_URL=https://api.schoolbus.example.com
UART_MASTER_PORT=/dev/ttyAMA0
UART_MASTER_BAUDRATE=115200
FACE_VECTOR_DIM=128
PRESENCE_CHECK_INTERVAL_SEC=180
LOG_LEVEL=INFO
DEVICE_PSK=b7e2f1a9c4d8...(hex 32 byte, sinh riêng cho từng xe, KHÔNG commit lên git)
SCHOOL_GEOFENCE_LAT=21.0021
SCHOOL_GEOFENCE_LON=105.8462
SCHOOL_GEOFENCE_RADIUS_M=100

```

`SCHOOL_GEOFENCE_*` dùng để nhận diện giai đoạn `ARRIVE_SCHOOL/DEPART_SCHOOL` ở mục 2.7(b).
`DEVICE_PSK` dùng để ký HMAC gói tin ở mục 4.0b — nên đưa vào `.gitignore`, không hard-code trong code.

6. SỬA LẠI CẤU TRÚC THƯ MỤC RASPBERRY PI

6.1. Vì sao cấu trúc hiện tại vừa thừa vừa thiếu

Thừa (nên bỏ hoặc gộp lại):

- `apps/gnss/` — GNSS **cắm vào STM32 Master** (UART2), Pi chỉ **nhận** dữ liệu GNSS đã qua Master relay, không tự đọc GPS. Không cần app driver phần cứng riêng trên Pi.
- `apps/sensor/` — cảm biến ghế và nhiệt độ/độ ẩm đều nằm ở STM32 (Slave/Master), Pi không đọc cảm biến trực tiếp. Giữ app này sẽ gây hiểu lầm là Pi có GPIO cảm biến.
- `models/` (nếu đang hiểu là ORM MySQL) — **MySQL chạy trên Server**, không chạy trên Pi. Pi không kết nối trực tiếp CSDL MySQL, chỉ trao đổi qua MQTT/API. Nếu để `models/` chứa ORM là sai vai trò gateway.

Thiếu (nên thêm):

- Không có module UART protocol hoá (khung frame ở mục 3) — hiện `communication` đang chung chung, chưa tách rõ **UART với Master** và **MQTT/API với Server**.
- Không có tầng **business logic / state machine** (đếm số lần sai mặt, debounce ghế 10s, điều kiện logout, so khớp học sinh-ghế) — nếu nhét hết vào `apps/monitoring` sẽ phình to, khó test.
- Thiếu `.env` — anh có tự nhắc trong mô tả nhưng cây thư mục chưa có.
- `models/` nếu bỏ ORM thì thiếu **domain model thuần Python** (Driver, Attendant, Student, Seat...) để dùng chung giữa các app.

6.2. Cấu trúc đề xuất

```

DoAnTotNghiep/
├── apps/
│   ├── authentication/      # đăng nhập, so khớp cục bộ, đếm mismatch, đếm absence
│   ├── camera/             # capture thread DUY NHẤT mở camera, đẩy frame vào
│   └── ──────────────────── buffer dùng chung (mục 4.3k) – login/logout, presence_monitor, stream ra Server đều
│   └── ──────────────────── chỉ ĐỌC từ buffer này, không tự mở camera riêng
│       ├── communication/
│       │   ├── uart_protocol.py    # đóng/mở khung frame với STM32 Master (mục 3)
│       │   ├── mqtt_client.py      # publish/subscribe với Server
│       │   ├── server_api.py       # fallback REST nếu cần (enroll, ảnh...)
│       │   ├── envelope.py         # đóng gói {type, ts, vid, data, sig} (mục 4.0/4.0b)
│       │   └── outbox_manager.py    # đọc/ghi hàng đợi ngoại tuyến local_cache.db (mục 7)
│       ├── presence_monitor/      # (đổi tên từ "monitoring") vòng lặp kiểm tra định kỳ
│       └── ──────────────────── 3 phút, debounce ghế 10s
│           └── ui/                # PyQt5
├── core/                        # business logic / state machine, KHÔNG đựng phần cứng
├── hay mạng
│   ├── session_manager.py        # điều kiện logout, trạng thái phiên làm việc
│   ├── boarding_logic.py         # so khớp RFID-ghế, học sinh cuối lên/xuống
│   └── seat_state.py              # debounce trạng thái ghế
├── models/                      # domain entity thuần Python (Driver, Attendant,
│   └── ──────────────────── Student, Seat, Vehicle)
├── schemas/                     # DTO (pydantic) cho payload MQTT & UART – đúng như
│   └── ──────────────────── anh đang làm
├── services/                   # hạ tầng dùng chung: logger.py, config_loader.py,
│   ├── checksum.py, hmac_signer.py (PSK)
│   └── data/

```

```

├── face_data/           # vector đặc trưng cục bộ (cache offline)
├── images/              # ảnh chụp tạm
├── logs/
├── local_cache.db       # (tùy chọn) SQLite – hàng đợi gửi khi mất mạng
├── tests/
├── .env
├── config.py            # đọc .env, expose settings dùng chung
├── main.py
├── README.md
└── requirements.txt

```

Tóm tắt thay đổi:

Hành động	Thư mục
 Bỏ	<code>apps/gnss/</code> , <code>apps/sensor/</code> (chuyển logic parse dữ liệu vào <code>communication/uart_protocol.py</code>)
 Đổi tên	<code>apps/monitoring/</code> → <code>apps/presence_monitor/</code> (rõ nghĩa: giám sát hiện diện, không phải dashboard)
 Thêm	<code>core/</code> , <code>.env</code> , <code>data/local_cache.db</code> (tùy chọn), tách rõ <code>uart_protocol.py</code> / <code>mqtt_client.py</code> / <code>server_api.py</code> trong <code>communication/</code>
 Giữ nguyên vai trò	<code>models/</code> giữ lại nhưng chuyển nghĩa: entity thuần Python, không phải ORM MySQL; <code>schemas/</code> , <code>services/</code> giữ nguyên đúng như anh thiết kế

7. HÀNG ĐỢI NGOẠI TUYẾN (OFFLINE QUEUE) — chỉ cache loại gửi được

Đúng như anh nói: **không phải type nào cũng cache được**, vì có loại cần phản hồi ngay để UI biết đường xử lý (login/logout), cache lại thì driver đứng chờ vô nghĩa. Chia rõ 2 nhóm:

7.1. Nhóm KHÔNG cache — cần kết nối trực tiếp (fire-and-wait)

`type 1,2,3,4,6,7,8,9,17,18,21` (login/logout/enroll của tài xế & phụ xe, và **xác nhận trạng thái phiên khi khởi động — type 21**) — nếu Pi phát hiện mất kết nối MQTT tại thời điểm request, **không gửi vào hàng đợi**, mà báo ngay lên UI: "Mất kết nối mạng, không thể xác thực lúc này" — để tài xế biết mà xử lý (thử lại, hoặc theo quy trình dự phòng riêng của trường nếu có).

7.2. Nhóm CÓ cache — không cần phản hồi ngay (fire-and-forget)

`type 5,10,11,12,13,14,19` — các sự kiện này chỉ cần **tới nơi**, không ai đang đứng chờ màn hình phản hồi. Toàn bộ nhóm này đi qua **outbox pattern**:

- Bảng `outbox` trong `data/local_cache.db` (SQLite):

Cột	Kiểu	Ghi chú
id	INTEGER PK	
type	INTEGER	
payload_json	TEXT	envelope đầy đủ (đã có sig)

Cột	Kiểu	Ghi chú
priority	INTEGER	0=thường, 1=cao (VD: seat_status), 2=khẩn (SOS/child_alone nếu lỡ mất mạng đúng lúc đó)
created_at	INTEGER	unix ts lúc tạo, không phải lúc gửi — để Server biết độ trễ thực tế
sent_at	INTEGER NULL	NULL = chưa gửi

- Quy trình: mọi sự kiện thuộc nhóm 7.2 **luôn ghi vào outbox trước**, rồi mới thử publish MQTT ngay. Publish thành công (broker ACK) → cập nhật **sent_at**, có thể dọn định kỳ các bản ghi đã gửi quá X ngày. Publish thất bại/mất mạng → giữ nguyên trong **outbox**, một luồng nền (background thread) cứ vài giây kiểm tra kết nối, hễ có mạng lại thì đẩy theo thứ tự **priority giảm dần, rồi created_at tăng dần**.
- Ngoại lệ: **emergency_sos** (15) và **child_alone_alert** (16) tuy về nguyên tắc là "khẩn cấp cần gửi ngay" nhưng **nếu đúng lúc đó mất mạng thì vẫn phải cache** (không được để mất luôn) — set **priority=2**, và luồng nền retry dày hơn hẳn (VD: mỗi 2 giây) cho riêng 2 loại này, không đợi chu kỳ retry bình thường.
- seat_status** (13) và **telemetry** (14): vì bản chất là dữ liệu theo dòng thời gian, khi offline lâu có thể sinh ra rất nhiều bản ghi tồn đọng — nên giới hạn: **chỉ giữ lại N bản ghi mới nhất** (VD: 500) trong outbox cho riêng 2 loại này, không giữ toàn bộ lịch sử, để tránh outbox phình to trên thẻ nhớ Pi khi mất mạng dài ngày.
- Gói tin gửi bù cần cờ riêng **is_replay: true** để Server không áp luật chống replay 30s (mục 4.0b) cho các gói tin này — Server dùng **created_at** gốc để ghi log đúng thời điểm xảy ra, không dùng thời điểm nhận được.

7.3. Ghi chú khác

- system/heartbeat**: giúp Server phân biệt "xe mất kết nối" với "xe không có sự kiện gì" — không đi qua outbox (không có ý nghĩa gửi bù).
- QoS MQTT**: QoS 1 cho sự kiện thường, QoS 2 cho **emergency_sos/child_alone_alert**.

8. CSDL SERVER (MySQL) — BẢNG **Vehicle** VÀ **TripSession** (mới bổ sung)

Nguyên tắc cốt lõi: **tách biệt hoàn toàn "phiên đăng nhập tài xế" (ai đang lái) khỏi "trạng thái chuyến đi" (xe đang ở giai đoạn nào, đã bao nhiêu học sinh)** — hai thứ này **không được gắn chung 1 bảng/1 vòng đời**, nếu không khi tài xế phải đăng nhập lại (VD sau mất điện, mục 2.9) sẽ vô tình làm trôi mất dữ liệu chuyến đi và gây báo động giả (ngghi bỏ quên trẻ) dù thực tế không có gì bất thường.

8.1. Bảng **Vehicle**

Cột	Kiểu	Ghi chú
vehicle_id (PK)	VARCHAR	biển số xe, khớp đúng vid dùng trong mọi payload MQTT
chassis_number	VARCHAR	mã khung xe
seat_count	INT	mặc định 16 — để mở rộng, không hard-code trong code Pi/Server
device_psk_hash	VARCHAR	lưu hash của PSK (mục 4.0b), không lưu plaintext
route_id (FK → Route)	INT	tuyến đang được gán cho xe
status	ENUM	active / maintenance / inactive

Cột	Kiểu	Ghi chú
created_at / updated_at	DATETIME	

8.2. Bảng **TripSession**

Đây chính là bảng lưu **trạng thái chuyến đi**, sống độc lập với phiên đăng nhập — Pi phải hỏi lại bảng này (không phải bảng đăng nhập) mỗi khi khởi động để đồng bộ đúng tiến trình chuyến, dù **session_state** ở mục 2.9 là **RESUMED** hay **REQUIRE_NEW_LOGIN**.

Cột	Kiểu	Ghi chú
id (PK)	BIGINT	
vehicle_id (FK → Vehicle)	VARCHAR	
route_id (FK → Route)	INT	tuyến thực tế chạy trong buổi này — không lấy mặc định từ Vehicle.route_id , vì cùng 1 xe có thể chạy tuyến khác nhau giữa sáng/chiều, hoặc đổi tuyến dự phòng khi cần (xe khác thay thế, tuyến gộp...)
trip_date	DATE	
trip_shift	ENUM	MORNING / AFTERNOON
trip_phase	ENUM	PICKUP / ARRIVE_SCHOOL / DEPART_SCHOOL / DROPOFF / COMPLETED (mục 2.7b)
students_onboard	INT	số học sinh đang ghi nhận trên xe — cập nhật theo sự kiện RFID/bulk, không phụ thuộc ai đang đăng nhập
boarded_student_ids	JSON	danh sách mã học sinh đã quét thẻ lên xe trong chuyến này (dùng để đối chiếu bulk alight/board ở mục 2.7b)
current_segment_order	INT	đoạn đường (RouteStop.stop_order) xe đang chạy tới — tăng lên khi GPS vào bán kính điểm cuối đoạn hiện tại (mục 8.5), bổ trợ song song với xác nhận RFID, không thay thế nhau — không còn cần thiết: với routing động (mục 8.5), "đã tới nơi" xác định trực tiếp bằng khoảng cách GPS tới RouteStop.lat/lon của điểm kế tiếp (suy ra qua boarded_student_ids), không cần đếm số đoạn riêng.
driver_id (FK, nullable)	VARCHAR	tài xế hiện tại của phiên đăng nhập — có thể đổi giá trị qua Agree/Disagree (mục 2.9) mà không ảnh hưởng các cột phía trên
attendant_id (FK, nullable)	VARCHAR	tương tự
updated_at	DATETIME	

Khoá duy nhất (unique key): (**vehicle_id**, **trip_date**, **trip_shift**) — đảm bảo 1 xe chỉ có đúng 1 dòng **TripSession** cho mỗi buổi mỗi ngày, bất kể tuyến (**route_id**) là gì. Nhờ vậy Pi khởi động lại chỉ cần tra đúng **vehicle_id + trip_date + trip_shift** (2 giá trị đầu tự suy ra từ đồng hồ hệ thống + **vid**) là lấy được đúng trạng thái, không cần biết trước **route_id**. Nếu thực tế đội xe của anh **cố định 1 xe luôn chạy đúng 1 tuyến duy nhất** (không đổi bao giờ) thì cột **route_id** này có thể bỏ, dùng thẳng

`Vehicle.route_id` cho gọn — em để sẵn ở đây vì phương án này vẫn đúng cho cả 2 trường hợp, xử lý sau này khi có xe dự phòng/đổi tuyến sẽ không phải sửa lại schema.

8.4. Bảng phục vụ tính "điểm đến tiếp theo" (`type=23`, mục 4.3.e2)

Để trả đúng `next_student_id/next_address` sau mỗi lần quét thẻ (đã loại học sinh nghỉ), Server cần thêm 2 bảng — không thuộc phạm vi Pi, chỉ ở phía Server:

Bảng	Cột chính	Ghi chú
RouteStop	route_id, student_id (INT, FK → <code>students.student_id</code>), stop_order, address, lat, lon (DECIMAL — toạ độ điểm đến/trả, do quản trị viên chọn bằng pin trên bản đồ, xem 8.4b), geometry (JSON toạ độ đoạn đường dẫn tới điểm dừng này — từ file OSRM/OpenRouteService tương ứng), distance_m, duration_s	Thứ tự dừng cố định của từng học sinh trong 1 tuyến (đặt sẵn khi tạo tuyến). geometry nạp 1 lần lúc setup tuyến từ các file JSON tải về (file0 → stop_order=1, file1 → stop_order=2...), không đọc file JSON thô lúc chạy thật. geometry/distance_m/duration_s tính không còn bắt buộc lưu sẵn — tuyến đường giờ được Server tính động mỗi lần quét thẻ dựa trên lat/lon này (xem 8.5 cập nhật); vẫn có thể giữ lại nếu muốn có bản đồ tổng quan tính lúc bắt đầu chuyến. student_id là INT khớp đúng khoá chính hiện có của bảng <code>students</code> (3, 4, 5, 7, 8...), không phải chuỗi tùy đặt
StudentAbsence	student_id (INT, FK → <code>students.student_id</code>), absence_date, shift	Phụ huynh đăng ký nghỉ qua app (mục 2.8, ý 3) — Server dùng bảng này để loại bỏ khi tính điểm dừng kế tiếp
School	school_id, name, address, lat, lon	Toạ độ trường — cũng chọn bằng pin trên bản đồ (8.4b), dùng làm điểm đến khi <code>is_last_student_picked = true</code> (mục 4.3.e2) và làm tâm geofence ARRIVE_SCHOOL/DEPART_SCHOOL (mục 2.7b). Nếu chỉ có 1 trường duy nhất, có thể gộp thẳng vào 1 dòng cấu hình chung thay vì tạo bảng riêng.

Khớp với DB thật đang chạy: bảng `students` đã có sẵn `rfid_code` (unique+index) — tra `type=11` (`student_scan`) chỉ cần 1 câu `WHERE rfid_code = ...`, không qua bảng trung gian nào. Bảng `students` cũng đã có sẵn `status` (text: "Đang trên xe" / "không ở trên xe") và `current_seat` — được `Backend/mqtt_listener.py` (dòng 218-232) cập nhật trực tiếp mỗi khi nhận `type=11: phase=1 (Pickup) → status="Đang trên xe", current_seat=seat_number; phase=4 (Dropoff) → status="không ở trên xe", current_seat=0`. **Cơ chế này đã đúng và đang chạy tốt, không cần thay** — chỉ bổ sung thêm 2 việc **tại đúng cùng vị trí code đó** (không tạo thêm điểm ghi dữ liệu rời rạc):

- Cùng lúc cập nhật `students.status`, cộng/trừ `TripSession.students_onboard` (+1 lúc Pickup, -1 lúc Dropoff) và thêm/xoá `student_id` khỏi `TripSession.boarded_student_ids` — để logic bulk alight/board (mục 2.7b) và cảnh báo 08/001 có dữ liệu tổng theo xe, vì `students.status` chỉ biết trạng thái **từng em**, không biết tổng số trên **1 xe cụ thể** tại 1 thời điểm.
- Về lâu dài (không gấp, chỉ khi anh có thời gian) có thể cân nhắc đổi `status` từ text tự do sang enum chuẩn (`BOARDED/ALIGHTED/ABSENT`) để tránh sai khác chính tả khi so sánh chuỗi ("Đang trên xe" phải khớp tuyệt đối) — nhưng đây là cải tiến thêm, không bắt buộc vì cơ chế hiện tại vẫn hoạt động đúng.

Logic tính `next_student_id`: lấy `RouteStop` theo `route_id` (từ `TripSession.route_id`) sắp theo `stop_order`, bỏ qua các `student_id` đã có trong `boarded_student_ids` (đã lên/xuống rồi) và bỏ qua các `student_id` có mặt trong `StudentAbsence` đúng ngày/buổi hôm đó → lấy phần tử đầu tiên còn lại. **Nếu không còn phần tử nào (đã đón hết) → đây chính là học sinh cuối cùng vừa quét → trả `is_last_student_picked = true` cùng toạ độ `School` thay vì `RouteStop` kế tiếp (mục 4.3.e2).**

8.4b. Chọn toạ độ trên bản đồ khi đăng ký học sinh/trường (mới)

Giao diện quản trị (Server) khi đăng ký `RouteStop` (học sinh) hoặc cấu hình `School` cần thêm bước chọn toạ độ bằng bản đồ, vì geocode tự động từ địa chỉ nhập tay (đặc biệt địa chỉ Việt Nam) thường lệch vị trí thực vài chục–vài trăm mét:

1. Admin nhập địa chỉ dạng text như bình thường → hệ thống geocode sơ bộ (Nominatim/Google Geocoding) để **định vị gần đúng vùng đó** trên bản đồ.
2. Bản đồ tự động phóng tới vùng gần đúng này, hiện 1 pin có thể **kéo/thả** để admin tự xác nhận đúng vị trí nhà/trường.
3. Khi admin xác nhận, toạ độ pin (`lat/lon`) được lưu vào `RouteStop/School` — đây là toạ độ **chính thức** dùng cho toàn bộ hệ thống (routing động, geofence trường), không dùng lại toạ độ geocode thô ở bước 1.

Về mặt kỹ thuật: đây là giao diện web thông thường (Leaflet/Google Maps JS + marker kéo-thả), không ảnh hưởng gì tới Pi/thiết bị trên xe — chỉ thay đổi phía Server/Dashboard.

8.5. Dữ liệu tuyến đường: bản đồ tổng quan tĩnh + routing động theo từng lượt quét thẻ (**cập nhật**)

Thay đổi so với bản trước: trước đây toàn bộ tuyến (từng đoạn `geometry`) được nạp tĩnh 1 lần từ file JSON lúc tạo tuyến, dùng lại y nguyên suốt chuyến. Giờ tách làm 2 việc độc lập:

Nạp 1 lần lúc tạo tuyến (không đọc lại lúc chạy thật): viết 1 script import (VD `services/route_loader.py`), đọc lần lượt từng file JSON (file0 = trường → nhà A, file1 = nhà A → nhà B, ..., file cuối = nhà E → trường), gán đúng theo `stop_order`, lưu `geometry/distance_m/duration_s` vào `RouteStop`:

Định dạng phổ biến nhất (OSRM/OpenRouteService) là `routes[0].geometry.coordinates` (mảng `[lon, lat]`) hoặc `routes[0].geometry` dạng chuỗi **encoded polyline** — cần giải mã trước khi lưu (thư viện `polyline` trong Python nếu là dạng chuỗi). Nếu file của anh là GeoJSON `LineString` riêng thì lấy thẳng `features[0].geometry.coordinates`:

1) Bản đồ tổng quan tĩnh (tuỳ chọn, chỉ để hiển thị lúc bắt đầu chuyến): nếu vẫn muốn vẽ sẵn toàn tuyến ngay khi **PICKUP** bắt đầu, Server có thể gọi 1 lần routing engine nối các điểm `RouteStop.lat/lon` theo `stop_order`, gửi Pi vẽ nền — đây chỉ là hình minh hoạ, không dùng để tính "đã tới nơi".

2) Routing động theo từng lượt quét thẻ (cơ chế chính, thay thế tuyến tĩnh cũ): mỗi khi học sinh quét thẻ thành công (type=11), Server gọi routing engine tính đường từ **vị trí GPS hiện tại của xe** (lấy từ telemetry gần nhất) tới **toạ độ điểm đến kế tiếp** (`RouteStop.lat/lon` của học sinh kế, hoặc `School.lat/lon` nếu `is_last_student_picked=true`), trả `next_route_geometry` trong payload type=23 (mục 4.3.e2). Pi nhận và **vẽ lại** đường này, thay cho đường cũ — tự thích ứng nếu xe đi lệch tuyến dự kiến, không cần tính trước toàn bộ tuyến từ đầu.

Dùng lúc chạy thật:

1. **Vẽ đường trên UI** — lúc bắt đầu **PICKUP**, Server gộp toàn bộ các đoạn `geometry` thành 1 đường liên tục gửi 1 lần cho Pi (không lặp lại mỗi lần quét thẻ), Pi vẽ tĩnh đường nền, chỉ cập nhật vị trí icon xe theo GNSS

thời gian thực (telemetry, type=14). Nay: Pi vẽ lại đường mỗi khi nhận next_route_geometry mới (sau mỗi lần quét thẻ), giữa 2 lần quét chỉ cập nhật icon xe theo GNSS (telemetry, type=14) trên nền đường đã có.

2. **Phát hiện "đã tới nơi" bằng GPS** — so khoảng cách GPS hiện tại với toạ độ điểm đến kế tiếp (RouteStop.lat/lon — thay cho current_segment_order + geometry tính cũ), vào bán kính ~30-50m thì coi là đã tới nơi. Nếu tới nơi mà chưa quét thẻ trong 1 khoảng thời gian nhất định → nhắc (giống 05/005 đã có) — **GPS chỉ để gợi ý, RFID mới là xác nhận chính thức**, hai cơ chế bổ trợ nhau chứ không thay thế.
 3. **Tính ETA cho app phụ huynh** — dùng duration_s trả về cùng lúc routing động (bước 2 ở trên) từ vị trí hiện tại tới nhà con họ, gửi thông báo "xe đang đến, còn khoảng N phút".
- Khi Pi gửi vehicle_status_request (type=21), Server trả về **đồng thời 2 phần độc lập**:
 1. session_state (0/1/2/3) — như mục 2.9 đã mô tả, chỉ nói về danh tính tài xế/phụ xe.
 2. trip_session hiện tại (trip_phase, students_onboard) — lấy từ bảng TripSession theo vehicle_id + trip_date + trip_shift, **gửi kèm luôn trong cùng payload type=22**, không cần request riêng.
 - Dù kết quả là RESUMED hay REQUIRE_NEW_LOGIN, Pi đều nhận đủ trip_phase + students_onboard để core/boarding_logic.py tiếp tục đúng mạch — **tài xế có thể phải đăng nhập lại (Disagree), nhưng số liệu học sinh/ghế không bị coi là "chuyến mới" và không kích hoạt nhằm cảnh báo 08/001**.

```
// Server → Pi, gộp cả session_state lẫn trip_session trong 1 payload type=22
{
  "type": 22, "ts": 1752720340, "vid": "29B12345",
  "data": {
    "session_state": 3,
    "trip_phase": "PICKUP",
    "students_onboard": 14,
    "boarded_student_ids": ["1A2B3C4D", "2A2B3C4D", "..."]
  },
  "sig": "..."}
}
```